

Unidade 02 | Capítulo 01



Fundamentos de POO

Prof.: Allberson Dantas



Sumário

- Classes e Objetos;
- Encapsulamento, Getters e Setters;
- Construtores e Sobrecarga.



O que é Programação Orientada a Objetos?

- A POO é um paradigma de programação que se baseia no conceito de "objetos" para estruturar e organizar o código;
- Cada objeto possui características e comportamentos, e esses são definidos por classes, que funcionam como moldes para a criação de objetos;
- Com a POO, conseguimos modelar o mundo real dentro do código.

Classes e Objetos

Classe: é uma estrutura que define as características (atributos) e comportamentos (métodos) de um tipo de objeto. É como um molde ou projeto.

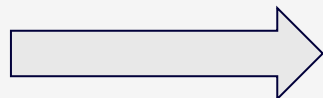
Mas o que é um **objeto**?

Classes e Objetos

Classe: é uma estrutura que define as características (atributos) e comportamentos (métodos) de um tipo de objeto. É como um molde ou projeto.

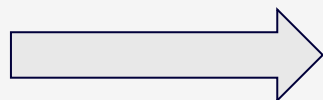
Mas o que é um **objeto**?

Podemos pensar em um carro:



Atributos: cor, modelo, ano.

Possui:



Comportamentos: acelerar, frear e buzinar

Classes e Objetos

Definição de Objeto: é uma entidade concreta, criada a partir de um molde chamado classe.

Em java o exemplo anterior seria feito da seguinte forma:
Primeiro criamos a classe Carro:

```
public class Carro {  
    String cor;  
    String modelo;  
    int ano;  
  
    void buzinar() {  
        System.out.println("Bii Bii!");  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex15ObjetoCarro.java

Classes e Objetos

Depois criamos um objeto:

```
public class Ex15ObjetoCarro {  
  
    public static void main(String[] args) {  
        Carro meuCarro = new Carro();  
        meuCarro.cor = "Vermelho";  
        meuCarro.modelo = "Sedan";  
        meuCarro.ano = 2022;  
  
        System.out.println("Meu carro é um " + meuCarro.modelo + " da cor " + meuCarro.cor);  
        meuCarro.buzinar();  
    }  
}
```

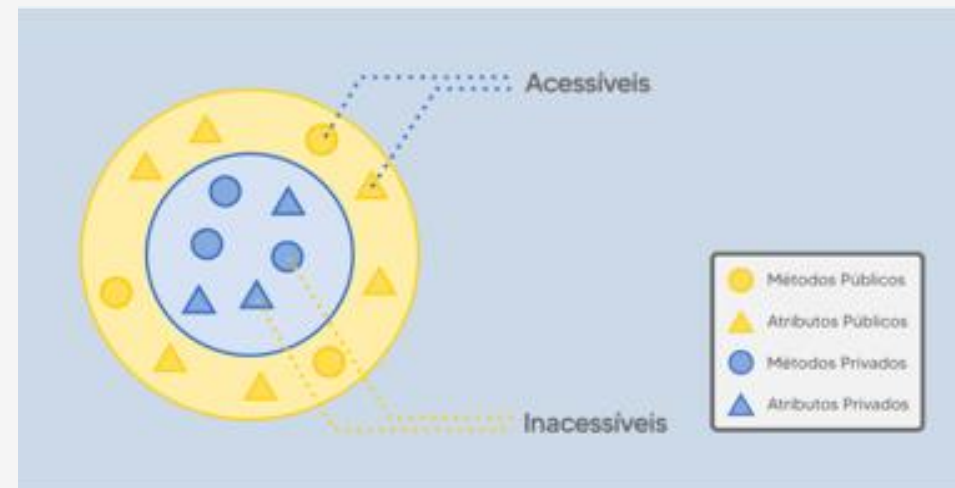
Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex15ObjetoCarro.java

Encapsulamento

O encapsulamento é a prática de ocultar detalhes internos de uma classe, expondo apenas o que é necessário para quem a utiliza.

Por que usar o encapsulamento?

Proteção dos dados;
Manutenção facilitada;
Controle de acesso.



<https://gerardao.com.br/2024/02/20/2-info-21-02-2024-poo/>

Encapsulamento

Utilizamos modificadores de acesso para definir o nível de visibilidade dos atributos e métodos:

public

private

protected

Getters e Setters

- No encapsulamento os atributos da classe são privados, ou seja, não podem ser acessados diretamente de fora da classe.
- Para permitir a leitura ou modificação desses atributos de maneira segura, usamos **Getters e Setters**.
- **Para acessar (get) e modificar (set).**

Exemplo: Encapsulamento, Getters e Setters

Agora veremos um exemplo que engloba Encapsulamento, Get e Set.

Primeiro criamos nossa classe Conta:

```
public class Conta {  
    private double saldo; // Atributo encapsulado  
  
    // Getter - Retorna o saldo atual  
    public double getSaldo() {  
        return saldo;  
    }  
  
    // Setter - Permite modificar o saldo de forma controlada (apenas positivo)  
    public void setSaldo(double saldo) {  
        if (saldo >= 0) { // Garante que o saldo nunca seja negativo  
            this.saldo = saldo;  
        } else {  
            System.out.println("Erro: O saldo não pode ser negativo!");  
        }  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex16Encapsulamento.java

Exemplo: Encapsulamento, Getters e Setters

Agora veremos um exemplo que engloba Encapsulamento, Get e Set.

Primeiro criamos nossa classe Conta:

```
// Método para realizar um depósito via setSaldo()
public void depositar(double valor) {
    if (valor > 0) {
        setSaldo(this.saldo + valor); // Chamamos setSaldo() internamente
        System.out.println("Depósito realizado: R$ " + valor);
    } else {
        System.out.println("Valor inválido para depósito.");
    }
}

// Método para realizar um saque via setSaldo()
public void sacar(double valor) {
    if (valor > 0 && valor <= saldo) {
        setSaldo(this.saldo - valor); // Chamamos setSaldo() internamente
        System.out.println("Saque realizado: R$ " + valor);
    } else {
        System.out.println("Saldo insuficiente ou valor inválido.");
    }
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex16Encapsulamento.java

Exemplo: Encapsulamento, Getters e Setters

Agora instanciamos um objeto Conta:

```
public class Ex16Encapsulamento {  
  
    public static void main(String[] args) {  
        // Criando um objeto da classe Conta  
        Conta minhaConta = new Conta();  
  
        // Definindo um saldo inicial usando setSaldo()  
        minhaConta.setSaldo(1000);  
        System.out.println("Saldo inicial: R$ " + minhaConta.getSaldo());  
  
        // Realizando um depósito  
        minhaConta.depositar(500);  
        System.out.println("Saldo após depósito: R$ " + minhaConta.getSaldo());  
  
        // Realizando um saque  
        minhaConta.sacar(200);  
        System.out.println("Saldo após saque: R$ " + minhaConta.getSaldo());  
  
        // Tentando realizar um saque maior que o saldo disponível  
        minhaConta.sacar(1500); // Isso deve exibir uma mensagem de erro  
  
        // Tentando definir um saldo negativo (validação do setSaldo)  
        minhaConta.setSaldo(-100); // Isso deve exibir uma mensagem de erro  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex16Encapsulamento.java

Construtores

- Um construtor é um método especial de uma classe que é chamado automaticamente quando um objeto é criado.
- Ele serve para inicializar os atributos da classe e pode conter lógica de configuração.

Regras dos Construtores:

- ✓ O nome do construtor deve ser igual ao nome da classe.
- ✓ Não possui tipo de retorno (nem void, nem outro tipo).
- ✓ Pode receber parâmetros para definir valores iniciais.

Construtores

Exemplo Prático com Construtores:

```
public class Pessoa {  
    String nome;  
    int idade;  
  
    // Construtor da classe Pessoa  
    public Pessoa(String nome, int idade) {  
        this.nome = nome; // "this" refere-se ao atributo da classe  
        this.idade = idade;  
    }  
  
    void apresentar() {  
        System.out.println("Olá, meu nome é " + nome + " e tenho " + idade + " anos.");  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex17Construtor.java

Construtores

Exemplo Prático com Construtores:

```
public class Ex17Construtor {  
  
    public static void main(String[] args) {  
        Pessoa pessoal = new Pessoa("Carlos", 25); // Usando o construtor  
        Pessoa pessoa2 = new Pessoa("Ana", 30);  
  
        pessoal.apresentar();  
        pessoa2.apresentar();  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex17Construtor.java

Sobrecarga de Construtores

A sobrecarga permite que uma classe tenha múltiplos métodos ou construtores com o mesmo nome, desde que tenham diferentes listas de parâmetros.

💡 Isso significa que podemos criar objetos de diferentes maneiras!

Sobrecarga de Construtores

Exemplo com Sobrecarga de Construtores:

```
public class Pessoa {  
    String nome;  
    int idade;  
  
    // Construtor que recebe nome e idade  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    // Construtor sobrecarregado que recebe apenas o nome  
    public Pessoa(String nome) {  
        this.nome = nome;  
        this.idade = 0; // Se a idade não for informada, definimos como 0  
    }  
  
    // Construtor padrão sem parâmetros  
    public Pessoa() {  
        this.nome = "Desconhecido";  
        this.idade = 0;  
    }  
  
    void apresentar() {  
        System.out.println("Olá, meu nome é " + nome + " e tenho " + idade + " anos.");  
    }  
}
```

Sobrecarga de Construtores

Exemplo com Sobrecarga de Construtores:

```
public class Ex18Sobrecarga {  
  
    public static void main(String[] args) {  
        Pessoa pessoa1 = new Pessoa("Carlos", 25); // Construtor com nome e idade  
        Pessoa pessoa2 = new Pessoa("Ana"); // Construtor com nome apenas  
        Pessoa pessoa3 = new Pessoa(); // Construtor sem parâmetros  
  
        pessoa1.apresentar();  
        pessoa2.apresentar();  
        pessoa3.apresentar();  
    }  
}
```

Material de Apoio: Para rodar o código, execute o arquivo da pasta Exemplos: Ex18Sobrecarga.java



Lista de Exercícios e Tarefas

Material de Apoio:

Lista de Exercícios: Lista 04 - Fundamentos de POO

Tarefas: TAREFA 04 - Fundamentos de POO



CAPACITA iRede

INOVAÇÃO E TECNOLOGIA

RESIDÊNCIA EM TIC 20

EXECUTOR



INICIATIVA



COORDENADORA PPI



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



*Este projeto foi apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991, no âmbito do PPI-SOFTEX, coordenado pela Softex e publicado Residência em TIC 20, DOU 01245.002631/2023-58.

